

Operational Deployment

From Seed to Autonomous Knowledge System

Registry: [\[HOWL-VDR-20-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-15-2026\]](#) → [\[HOWL-VDR-16-2026\]](#) → [\[HOWL-VDR-17-2026\]](#)

→ [\[HOWL-VDR-18-2026\]](#) → [\[HOWL-VDR-19-2026\]](#) → [\[HOWL-VDR-20-2026\]](#)

DOI: 10.5281/zenodo.20241035

Date: May 2026

Domain: Applied Philosophy / Computational Linguistics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

VDR-19 established that a hybrid LLM-integer-Prolog architecture self-extends through usage — each session deposits persistent Prolog rules, Python scripts, and provenanced facts that compose with prior accumulated state. VDR-20 specifies how to deploy that architecture as a running system. Four prompt runner types — interactive, polling, processor, and internal processing — share identical VDR infrastructure differentiated only by trigger pattern and grant scope. An owner-local filesystem interface provides directories for data ingress, task specification, configuration, output, and review. A coverage loop driven by VDR's exact Remainder arithmetic converts topic specifications into measurable gap descriptions, autonomously fetching and compacting documents until coverage targets are met. The practical on-ramp is local directories: Project Gutenberg, man pages, source repositories, and language documentation exercising every component of the compaction pipeline with zero external dependencies. The owner directs the system using the full tool stack — VDR interactive chat, conventional LLMs for planning, web search, manual configuration — with every decision becoming facts and rules that prompt runners execute. All runner types, directory interfaces, task pipelines, and coverage mechanisms operate through the same primitive pipeline governed by the same visibility, scope, grant, and audit model specified in VDR-16. This paper introduces no new primitives, builtins, struct fields, or modules.

1. From Architecture to Running System

VDR is a hybrid architecture where a large language model generates only judgment tokens and prose tokens — roughly five to twenty percent of what a conventional LLM would produce for the same task. The remaining eighty to ninety-five percent of conventional token output — arithmetic, formatting, state reconstruction, deduction, hedging — is handled by exact integer primitives operating on data stored in knowledge bases at integer addresses. The LLM emits structured command tokens, approximately eight per primitive invocation, drawn from a vocabulary of around three hundred primitive names and two hundred data paths. These command tokens have approximately six bits of entropy per token compared to fifteen bits for full-vocabulary generation, making them low-entropy, low-error, and cheap.

Data in this architecture never flows through the LLM’s token stream. External data enters through primitives — API fetches, file reads, document parsers — and is stored as facts at integer addresses in knowledge bases. The LLM references data by typed dotted paths and integer identifiers. Primitives operate on data at those addresses. Results are written to new addresses. The LLM receives typed summaries. Output is generated through grammar templates that pull values from knowledge base addresses into formatted content slots.

VDR-19 established that this architecture self-extends. The LLM doesn’t just call primitives — it writes new Prolog rules that become immediately available to VDR’s frontier-based evaluator, writes Python scripts that execute in sandboxed containers and persist as reusable analytical capabilities, and writes new grammars and compaction rules that handle future documents of the same type without LLM involvement. Each session leaves behind persistent, inspectable, reversible infrastructure that all future sessions within scope can query and build upon.

VDR-20 answers the operational question that VDR-19 leaves open: how does this actually run? VDR-19 describes self-extension driven by human usage — a person asks questions, the system accumulates. But a system that only grows when humans drive it is limited by human attention. VDR-20 specifies the runner architecture that makes accumulation autonomous, the filesystem interface that makes the owner’s life practical, and the coverage loop that makes self-training directed rather than random.

The core insight is simple. The system runs multiple LLM instances, all sharing identical VDR architecture — same command tokens, same primitives, same scoped knowledge base access, same grant model, same audit trail. The instances differ only in what triggers them and what grants they hold. Some wake when a human types. Some wake on a timer. Some maintain persistent data connections. Some examine the system’s own state. They all deposit into shared knowledge bases through the same primitive pipeline. The same security, provenance, and alignment guarantees from VDR-16 and VDR-17 apply identically to every instance because every instance operates through the same structural mechanisms.

2. Prompt Runner Architecture

A prompt runner is an LLM instance bound to a VDR session. Every runner has a session identity, a position in the knowledge base scope tree, a set of grants authorizing specific operations, and access to the knowledge bases visible from its scope position. What differentiates runner types is not their architecture but their trigger pattern — what causes them to activate — and their grant scope — what operations they are authorized to perform.

Interactive Runners

Interactive runners are the human-facing layer. One instance per user session. The runner activates when the user provides input — a chat message, a document upload, a query, a project command. The user communicates in natural language. The runner's LLM interprets intent, emits command tokens to invoke primitives, writes rules, stores findings, and generates output through grammar templates. Each interaction deposits facts and rules into the user's project knowledge bases through the standard VDR pipeline.

Interactive runners are reactive. They wake on input and produce output. Between interactions they are idle, consuming no compute. Their grants are inherited from the authenticated user — the same visibility and scope restrictions that govern the user's data access govern the runner's operations. An interactive runner for an engineering user cannot access HR knowledge bases for the same structural reasons the user cannot: scope checks on integer values set at authentication.

Polling Runners

Polling runners are timer-driven. A scheduler spawns a polling runner every N seconds, where N is configurable per poller. The interval can be milliseconds for high-frequency operational monitoring, seconds for routine task dispatch, minutes for periodic maintenance, or hours for daily batch operations.

On each cycle the poller checks its assigned watch targets: queues, counters, trigger conditions, and directory watch lists. If a queue has items, the poller dequeues and processes or routes them. If a counter has crossed a threshold, the poller fires the associated rules. If a scheduled task is due, the poller initiates it. If a watched directory contains new files, the poller enqueues compaction tasks for them.

Pollers are the system's heartbeat. They don't wait for humans. They find work and do it. Their grants are system-level: queue management, directory watching, task dispatch. They do not hold user-level data access grants — they route work to runners that do.

Each polling cycle is a short-lived operation. The poller spawns, runs one cycle, and terminates. A fresh instance spawns for the next interval. This means every poller cycle operates with a fresh LLM in its optimal attention window, inheriting all accumulated knowledge through integer-addressed knowledge base access. There is no attention degradation over long polling lifetimes because there are no long polling lifetimes.

Processor Runners

Processor runners maintain persistent connections to data sources. An open stream to a metrics API, an active webhook listener, a file watcher on a streaming data directory, a subscription to an event feed. Data arrives continuously and the processor compacts and stores it in real time.

A processor watching a Prometheus metrics stream deposits time-series facts continuously into the appropriate knowledge base. A processor connected to a document feed compacts entries as they arrive. A processor watching an upload directory processes files as they appear.

Processors are long-lived and data-driven. Unlike pollers, they don't terminate after each unit of work — they maintain their connection and process data as it flows. To prevent LLM attention degradation over long sessions, processors periodically snapshot their state and respawn as fresh clones. The snapshot captures all accumulated session context as knowledge base facts. The new clone inherits those facts at integer addresses and resumes processing with a fresh LLM. The data stream continues uninterrupted because the connection state is managed at the infrastructure level, not in the LLM's context window.

Processor grants are specific to their data source: credential grants for their external connection, plus write grants for their target knowledge bases. A processor watching metrics cannot write to the legal knowledge base. A processor ingesting documents cannot issue API calls to external services. Each processor has the minimum grants necessary for its function.

Internal Processing Runners

Internal processing runners are self-directed. They activate on their own schedule — typically every few minutes to every few hours depending on the evaluation they perform — to examine the system's internal knowledge base state.

On each cycle an internal processor evaluates rule sets across knowledge bases within its read scope. It runs consistency checks — do facts from different sources contradict? It computes derived facts from newly accumulated data — aggregations, trend computations, relationship inferences that follow from existing rules applied to new facts. It identifies coverage gaps — topics where the owner specified depth targets that the current knowledge base doesn't meet. It computes coverage metrics where the Remainder is an operational description of what's missing.

Internal processors are the system's self-reflection. They don't respond to external events. They examine what the system knows and improve it. Their grants are broad read access across project knowledge bases (to evaluate coverage and consistency) plus write access for derived facts, coverage metrics, and gap descriptions. They cannot modify source facts — only the original ingestor or the owner can do that.

Like pollers, internal processors are short-lived per cycle. Spawn, evaluate, write findings, terminate. Fresh LLM every cycle.

What All Runners Share

Every runner type operates through the same primitive pipeline. Every command token invocation goes through grant verification before execution. Every knowledge base access goes through visibility and scope checks. Every fact written carries provenance — session identity, user identity, turn number, source references. Every operation is logged in the append-only audit knowledge base.

The differentiation between runner types is entirely in trigger pattern and grant scope. The architecture doesn't distinguish between a fact written by an interactive runner responding to a human and a fact written by an internal processor evaluating coverage. Both are knowledge base facts at integer addresses with provenance. Both are subject to the same visibility, scope, and constraint rules. Both are queryable, retractable, and auditable through the same mechanisms.

3. The Owner-Local Interface

The owner is the person or team who deploys and directs the VDR system. They need a practical way to feed data in, specify tasks, configure the system, receive output, and review proposals. The owner-local interface is a set of filesystem directory conventions watched by polling runners.

Directory Structure

The owner's local filesystem has designated directories that the system monitors through polling runners with appropriate filesystem grants.

The **ingress directory** is where the owner drops files for compaction. Any file placed here — a PDF, a markdown document, a code repository, a CSV dataset, a collection of text files — gets picked up by a poller on its next cycle and enqueued for processing. A processor runner compacts the file according to accumulated compaction rules and grammars, storing the resulting facts in the appropriate knowledge base based on classification rules. If the file format is novel — not covered by any existing compaction rule — the LLM writes a new compaction rule, which then handles all future files of that type without LLM involvement.

The **tasks directory** is where the owner drops task specifications. A task is a file describing work the system should do. “Fetch the Python asyncio documentation and compact it.” “Reorganize the networking knowledge base into protocol-level children.” “Run coverage analysis on the Zig standard library topic.” “Generate a cross-reference report between the Linux kernel memory management source and the relevant man pages.” A poller picks up task files, the system compacts them into executable command sequences (they're structured text, handled like any other document), and routes them to the appropriate runner type for execution.

The **config directory** is where the owner places configuration changes. Hierarchy proposals as structured facts. Scope adjustments. Coverage targets for topics and depth levels. Grant specifications for new runners or data sources. Policy changes. These are parsed

into knowledge base facts and rules that take effect on the next evaluation cycle. The system doesn't need to restart or reload — facts asserted into knowledge bases are live immediately.

The **output directory** is where the system deposits results. Generated reports, exported data files (CSV, JSON, formatted documents), coverage metric summaries, and completed task outputs. The owner checks this directory on their own schedule.

The **review directory** is where the system places items needing owner judgment. Proposed hierarchy changes the system is uncertain about. Novel document classifications where rules conflict. Coverage decisions where multiple strategies are possible. Anomalies flagged during consistency checking. Each review item includes the system's proposed action, its reasoning as a provenance chain, and the specific question it needs answered. The owner reviews, approves, modifies, or rejects. Their decision becomes facts and rules that handle similar situations in the future.

How Directory Watching Works

A polling runner is configured with a filesystem grant for each watched directory. On each cycle it lists directory contents and compares against a manifest of already-processed files stored as knowledge base facts. New files are detected by their absence from the manifest. Each new file is enqueued as a compaction task with metadata: source path, file type (detected by extension and content inspection), timestamp, and size.

The enqueued task goes through the same task pipeline as any other task — validation against the requesting identity's grants, priority ordering, assignment to an appropriate runner. The polling runner that detected the file doesn't process it directly. It enqueues and moves on. A processor runner picks up the compaction task and does the work.

Processed files are recorded in the manifest with their provenance: when processed, which compaction rule was used, which knowledge base received the facts, how many facts were extracted. The owner can query this manifest to see what's been processed and trace any fact back to its source file.

File Type Routing

When a new file is detected, the system determines how to handle it based on accumulated classification rules. A Python source file routes to the programming knowledge base branch with Python-specific compaction rules. A markdown document is classified by content — if it contains API documentation patterns it routes differently than if it contains narrative prose. A PDF is processed through document extraction primitives before compaction.

Initially the classification rules come from seed layer three. As the system processes more documents, it writes more specific classification rules. After processing fifty Python files and thirty Zig files, the system has rules that distinguish language-specific patterns and route accordingly. After processing documentation, source code, and tutorial content, it has rules that classify by content type regardless of file format. These classification rules accumulate and amortize like every other rule in the system.

4. The Interactive Interface

Beyond the filesystem, the owner and users interact with the system through chat sessions and programmatic API access.

Chat Interface

The owner or user connects to an interactive runner through a chat session. They describe what they want in natural language. The LLM interprets intent and translates it into structured operations — the thing LLMs are genuinely good at.

The owner sits down and says: “I’ve got Gutenberg in /data/gutenberg, man pages in /usr/share/man, Linux source in /data/repos/linux, Python and Zig repos here and here. I want programming as one branch, literature as another, OS internals as a third. Programming should have per-language children. Literature should organize by period and genre. OS internals should cross-reference with the man pages and the programming branch where APIs surface.”

The LLM produces a proposed hierarchy as Prolog facts. Knowledge base tree structure, scope relationships, visibility defaults, cross-reference rules. The owner looks at it. “Move the networking stuff out of OS internals into its own branch — it cross-cuts everything.” The LLM adjusts the hierarchy facts. Three minutes of conversation and the organizational plan is a set of knowledge base facts and rules ready to execute.

The owner says “run it.” The polling runners start walking directories on their next cycles. The hierarchy is live. The coverage targets are active. The system is working.

The chat interface is also where the owner does ongoing management. “What’s the coverage on Python standard library?” The system queries coverage metrics and reports with specific gap descriptions. “Go deeper on async programming.” A new coverage target is asserted. The polling runners will pick up the gap on their next cycle. “Show me everything we know about mmap across all sources.” A cross-reference query across the OS internals, man pages, and programming branches returns a unified view with provenance showing which facts came from kernel source, which from man pages, which from documentation.

API Interface

Programmatic access to the same VDR primitive pipeline. External systems submit queries, assert facts, enqueue tasks, and read results through a structured API. Each API session has its own identity, scope, and grants — the same security model as every other access path.

The API enables integration with CI/CD pipelines that deposit build results as knowledge base facts, monitoring systems that query coverage metrics, external tools that enqueue analysis tasks, and other automated systems that read from or write to the VDR knowledge base. Every API operation goes through the same grant verification, scope checking, and audit logging as every other operation in the system.

Planning with External Tools

The owner can use any tool for thinking and planning. Claude, GPT, a local model, a text editor, pen and paper. The output becomes input to the VDR system through whatever channel is convenient — a file dropped in the tasks directory, a message in the chat interface, a structured document in the config directory.

The owner might use a conventional LLM to brainstorm a knowledge organization strategy, write up the result as a markdown file, and drop it in the tasks directory. The VDR system compacts the plan into executable rules. The owner might do web searches for best practices on organizing programming language knowledge bases, save the articles locally, and drop them in the ingress directory. The system compacts those articles — now the organizational strategy itself is informed by accumulated knowledge about knowledge organization.

VDR doesn't care how the owner arrived at a decision. It cares that the decision becomes facts and rules it can execute. The boundary between planning and execution is just whether the output is a conversation or a set of asserted facts. Both are useful. Both feed the same system.

5. The Coverage Loop

The coverage loop is the mechanism that converts self-extension from passive accumulation into directed self-training. Without the coverage loop, the system accumulates whatever it encounters. With the coverage loop, it identifies what it's missing and goes looking for it.

Topic Specification

The owner specifies topics and depth targets. These become knowledge base facts with concrete, measurable structure. A specification like “programming — Python and Zig, API coverage, intermediate competency” becomes:

Facts declaring the topics: Python under programming, Zig under programming. Facts declaring the depth target: intermediate. Rules defining what intermediate means for a programming language: API surface area above a threshold percentage, concept relationships mapped between core modules, common patterns encoded as rules, error handling documented for standard operations.

These aren't vague goals. They're measurable conditions against the knowledge base state. How many facts exist for Python's standard library modules? How many relationship rules connect modules to each other? How many compaction rules cover Python documentation formats? Each question has an exact integer answer.

Coverage Evaluation

Internal processing runners evaluate knowledge base state against coverage rules on their schedule. The evaluation produces a coverage metric as a VDR quantity — a value with a denominator and a Remainder.

The value is what's covered. The Remainder is what's missing. Critically, the Remainder is not a percentage or a progress bar. It is a typed, decomposable, actionable description of specific gaps. Not “13% uncovered” but a structured list: asyncio event loop patterns absent, typing module generics absent, dataclass inheritance patterns absent. Each gap is a queryable fact in the coverage knowledge base with enough specificity to be converted into a fetch task.

This is the Remainder doing real work. In VDR arithmetic, the Remainder is the exact leftover from integer division — never discarded, never rounded, always preserved as a first-class part of the result. In coverage evaluation, the Remainder of the coverage metric preserves exactly what the system doesn't know yet. The system's ignorance is queryable in the same way its knowledge is.

Gap-to-Task Conversion

Polling runners read coverage metrics on their cycle. When they find gaps — Remainders with actionable content — they convert each gap into a fetch task. Missing Python asyncio coverage becomes a fetch task targeting asyncio documentation. Missing cross-reference between generators and coroutines becomes a targeted query looking for content that connects these concepts. Missing error handling patterns for the json module becomes a fetch task for json module examples and error documentation.

The fetch tasks go into the task queue with metadata: which coverage gap they address, what priority (derived from the coverage rules and the owner's depth targets), and what success looks like (which specific coverage conditions would be satisfied by the fetched content).

Fetch, Compact, Re-evaluate

Processor runners execute fetch tasks. For local directory sources, this means reading files from the relevant directories. For network sources (when granted), this means retrieving URLs. For owner-provided sources, this means processing files from the ingress directory.

Fetched documents are compacted through the standard pipeline: format detection, grammar-based structural extraction, entity identification, relationship mapping, fact storage with full provenance. The resulting facts are deposited in the appropriate knowledge bases based on classification rules.

On the next cycle, internal processors re-evaluate coverage. The new facts are in the knowledge base. Coverage rules fire against the updated state. Remainders shrink where the new facts addressed the gaps. New gaps may appear — the system discovers connections it didn't know to look for. Processing asyncio documentation reveals that asyncio interacts with the threading module in ways the system hasn't covered. A new gap appears. A new fetch task will be generated.

The cycle continues until coverage targets are met or the owner adjusts targets. There is no terminal state — the owner can always deepen coverage, broaden scope, or add new

topics. The system adapts because every target change is just a fact assertion that the internal processors pick up on their next cycle.

Meta-Rule Accumulation

After processing enough documents in a domain, the system writes rules that generalize beyond specific content. After compacting Python, Zig, and Rust documentation, the LLM writes rules about how programming language documentation is structured in general. API reference sections have this shape. Tutorial content has this shape. Error documentation has this shape. Type system descriptions have this shape.

These meta-rules accelerate future domain ingestion. When the owner adds a new programming language, the system already knows the documentary structure. Compaction rules for the new language build on the generalized patterns, requiring LLM judgment only for what's genuinely different about this language's documentation.

The same generalization happens across domains. After processing programming documentation, legal contracts, medical papers, and SRE incident reports, the system has meta-rules about document structure in general — how structured reference material differs from narrative exposition, how cross-referencing works across document types, how hierarchical organization maps to knowledge base scope trees. These meta-rules are knowledge about knowledge — they accelerate every future ingestion task regardless of domain.

6. Local Directory Bootstrap

The fastest path from a seeded system to a useful knowledge base is local directories. No API keys, no rate limits, no network latency, no authentication complexity, no external dependencies that could fail or change. Point pollers at directories and let the pipeline run.

Why Local First

Local directories serve as both the practical on-ramp and the validation path. If the system can't compact a man page from local disk — one of the most structured, predictable document formats in computing — it can't handle a live API stream or an unstructured web page. Local I/O isolates the compaction pipeline from every variable except the pipeline itself.

Local bootstrapping also lets the owner build confidence. They watch the system process documents they know well. They check the extracted facts against their own understanding. They verify that the cross-references make sense. They correct organizational decisions while the stakes are low and the data is familiar. By the time the system is processing unfamiliar content from external sources, the owner trusts the pipeline because they've seen it work on known material.

Project Gutenberg

Project Gutenberg texts are uniform in format, massive in volume, and deeply cross-referenced in content. A poller walks the /data/gutenberg directory tree. For each text file it enqueues a compaction task.

The first few books require LLM judgment for the compaction rule. Gutenberg has a standard header format — title, author, release date, language, character encoding, a preamble, and standardized section markers. The LLM identifies this structure, writes a compaction rule for Gutenberg headers, and writes grammars for the body content. After processing perhaps five to ten books, the header compaction is mechanical — the rule handles structural extraction without LLM involvement.

Body compaction requires ongoing LLM judgment because literary content is semantically rich. Characters, locations, events, themes, narrative structure, dialogue patterns — these are extracted through LLM judgment calls about what entities and relationships matter. But the extraction becomes faster as rules accumulate. After processing several novels, the system has rules about how novels introduce characters (appearance in early chapters with descriptive passages), how plot events chain (temporal and causal markers), how themes recur (repeated imagery and vocabulary clusters). These rules don't eliminate LLM judgment but they focus it — the LLM confirms or adjusts what the rules propose rather than starting from nothing.

After a hundred books, the system has a literature knowledge base with cross-references between authors, periods, styles, and influences. A query about narrative techniques returns facts drawn from dozens of novels with provenance tracing each observation to its source text, chapter, and extraction rule.

Unix Man Pages

Man pages are among the most predictable document formats in existence. NAME, SYNOPSIS, DESCRIPTION, OPTIONS, EXAMPLES, SEE ALSO — the sections are standardized and the content within each section follows consistent patterns. The compaction rule for man pages gets written on the first page or two. After that, structural extraction is entirely mechanical.

The value of man pages isn't in the individual page facts — it's in the cross-referencing. The SEE ALSO section is explicit relationship data. Every man page declares what other commands, functions, or concepts it relates to. The system extracts these as relationship facts directly: ls relates to dir, grep relates to regex and sed and awk, find relates to xargs and locate. After processing the full man page collection, the system has a comprehensive command and function relationship graph built entirely from the documents' own declared relationships.

The SYNOPSIS sections provide structured API facts — function signatures, parameter types, return types, option flags. These compose with the programming knowledge base. The C library functions documented in man section 3 relate to the same functions implemented in the Linux source and referenced in programming documentation. The

cross-referencing rules connect these automatically once the facts exist in scope-accessible knowledge bases.

Source Code Repositories

Source code repositories are the richest and most complex data source. The Linux kernel repository alone contains C source files with code and comments, header files with type definitions and API declarations, Makefiles with build dependency information, Kconfig files with configuration option hierarchies, and documentation directories with structured explanatory text. Each file type needs different compaction rules.

The density of relationships in source code is high. A function in `mm/mmap.c` relates to a data structure defined in `include/linux/mm_types.h`, which relates to the `mmap(2)` man page, which relates to virtual memory concepts documented in OS textbooks. The cross-referencing rules that connect these exist at different scope levels — source-internal references at the project level, source-to-manpage references at the OS internals level, source-to-textbook references at the programming level.

Processing a large source repository exercises every component of the system. Multiple compaction rule types are needed for different file formats within the same repository. Cross-referencing is dense and multi-level. The hierarchy must accommodate both the filesystem layout of the repository and the conceptual organization of the codebase. Classification rules must distinguish between implementation files, interface files, build files, configuration, and documentation.

After processing the Linux kernel, a language implementation (Python or Zig source), and a few well-structured open source projects, the system has compaction rules for every common source file type, cross-referencing rules for code-to-documentation relationships, and meta-rules about how source code repositories are organized in general.

API Documentation and Language References

Language documentation — Python standard library reference, Zig standard library documentation, language specifications — is structured reference material. It exercises per-language knowledge base organization, API surface coverage metrics, and cross-language comparison rules.

After processing documentation for two or three languages, the system writes generalized rules about how language documentation is structured. Function signatures follow patterns. Type descriptions follow patterns. Module organization follows patterns. These generalized rules accelerate processing of any subsequent language documentation.

Coverage metrics are particularly meaningful for API documentation because completeness is measurable. The Python standard library has a finite number of modules. Each module has a finite number of public functions. Each function has a signature, a description, and example usage. Coverage can be measured as the fraction of this surface area that exists as knowledge base facts, with the Remainder identifying exactly which modules, functions, or aspects are missing.

The Hierarchy Problem

The system generates an initial knowledge base hierarchy reflecting the structure it encounters — one branch per data source, sub-branches per directory or category within each source. But filesystem layout doesn't match how the owner wants to search or how they want security scoped.

All C-related content — man page function documentation, kernel source comments, the C language specification, C programming tutorials from Gutenberg — should be queryable together even though it came from four different filesystem paths and four different compaction pipelines. The kernel internals knowledge base might need owner-only visibility while the man pages knowledge base is public. Programming languages should be siblings under a common parent so cross-language comparison rules compose naturally, but literature should be in a separate top-level branch so a query about Python the programming language doesn't surface results about Monty Python's Flying Circus.

The system proposes a hierarchy based on its accumulated classification and organizational rules. Initially these proposals are rough — the classification rules are thin and the system doesn't yet know the owner's preferences. The owner reviews through the chat interface or by examining proposals in the review directory. They approve, modify, or reject. Each decision becomes facts and rules: "networking content goes in its own branch, not under OS internals." "Cross-reference programming APIs with their man pages." "Keep literature period-organized at the top level."

These organizational rules accumulate like every other rule. After the owner has corrected ten hierarchy proposals, the system's eleventh proposal reflects those corrections. After fifty corrections, the system's proposals largely match what the owner would choose. The organizational rules are knowledge base facts subject to the same amortization as every other rule — expensive to establish through initial owner interaction, nearly free once established.

What You End Up With

After the system has consumed a meaningful local library, the owner has a knowledge base tree where every fact is at an integer address with full provenance tracing back to the source file, the directory path, and the specific compaction rule that extracted it. Cross-reference rules connect concepts across sources — the same algorithm described in a textbook, implemented in kernel source, documented in a man page, and referenced in a Gutenberg-era mathematics text. Coverage metrics with operational Remainders identify exactly what's thin and what's well-covered. Compaction rules for every document format encountered are ready to handle new documents of the same types without LLM judgment. Meta-rules about document structure in general accelerate processing of formats the system hasn't seen yet.

And every LLM instance that did this work has already terminated. Each one ran its cycle and exited. The knowledge is in the knowledge bases, not in any model's weights or context window. The next instance to query this knowledge base will be fresh, operating in its optimal attention window, with access to everything that every prior instance

accumulated.

7. The Owner's Role

The owner is not a system administrator monitoring processes. The owner is a director using every available tool to shape what the system becomes.

Planning with Any Tool

The owner can use the VDR interactive chat to plan — describe goals in natural language and let the LLM produce hierarchy proposals, coverage targets, and organizational rules as Prolog facts. They can use a conventional LLM outside VDR entirely — brainstorm with Claude or GPT, think through organizational strategies, draft hierarchy proposals in markdown — and then feed the result into VDR through the tasks directory or config directory. They can do web searches for best practices on knowledge organization, save articles locally, drop them in the ingress directory, and let the system compact the articles into knowledge about knowledge organization.

The planning tool is irrelevant. What matters is that every decision, however it was reached, becomes facts and rules that the prompt runners execute. A hierarchy proposal drafted in a conversation with GPT and saved as a markdown file becomes exactly the same Prolog facts as a hierarchy proposed by the VDR interactive chat. The system doesn't distinguish between them at the operational level — both are knowledge base facts with provenance.

Judgment at Chosen Granularity

The owner can micromanage every knowledge base placement — reviewing each compaction result, approving each hierarchy decision, specifying exactly where each cross-reference rule should apply. Or they can set high-level policies — “programming languages as siblings, literature separate, cross-reference APIs with man pages” — and let the system fill in every detail, bringing them only exceptions it can't resolve from existing rules.

Both styles produce the same thing: facts and rules in knowledge bases. The system adapts to the owner's style because it doesn't mechanically distinguish between a specific instruction (“put mmap documentation in OS internals under memory management”) and a general policy (“organize OS internals by subsystem”). Both become Prolog facts. The specific instruction handles one case. The general policy handles many cases through rule matching. The owner works at whatever level of detail they find productive.

Shifting Over Time

Early in the system's life, the owner is more active. They set the initial hierarchy. They define coverage targets. They correct organizational proposals while the classification rules are thin. They point pollers at new directories and configure processor connections. They're directing.

As the system accumulates organizational rules that reflect the owner’s preferences, the owner’s role shifts. The system’s hierarchy proposals match what the owner would choose. Coverage targets are met or self-adjusting based on rules the owner approved earlier. Classification rules handle new document types based on generalized patterns. The owner shifts from directing to auditing — checking that what the system learned is correct, occasionally correcting a misclassification, adjusting depth targets as priorities change.

The Owner Never Loses Control

Every fact, every rule, every hierarchy decision, every organizational choice is inspectable. The owner can query the provenance chain of any fact — where it came from, which compaction rule extracted it, when it was ingested, what source file it traces to. They can examine any rule — what it does, when it was written, what session and what input prompted it. They can retract anything that’s wrong, and the system adjusts cleanly because retraction is a first-class operation in VDR. Retracting a fact removes it from query results. Retracting a rule removes its inferences from future evaluations. The Remainder of affected coverage metrics updates to reflect what was removed.

This isn’t trust. The owner doesn’t need to trust the system because they can verify everything it did. The audit trail is complete — every access through the primitive pipeline is logged, every fact carries provenance, every rule carries provenance, every organizational decision is traceable. The system trains itself, but the owner sees exactly what it learned, why it learned it, from what source, and can undo any of it surgically.

8. Thread Specification

Thread Lifecycle

Each runner type has a distinct lifecycle pattern, but all share the same session establishment process: acquire session identity, resolve scope position, load grants, connect to visible knowledge bases.

Interactive runners are spawned when a user connects through the chat or API interface. The session persists for the duration of the user’s connection. On disconnect or idle timeout, the session terminates. Any session state that wasn’t explicitly promoted to project-level knowledge bases is discarded. The LLM instance is released.

Polling runners are spawned by a system scheduler at their configured interval. The scheduler maintains a registry of pollers: which directories to watch, which queues to check, which trigger conditions to evaluate, and at what frequency. On each interval the scheduler spawns a fresh runner with system-level grants. The runner executes one cycle — checking all its assigned watch targets and dispatching any work it finds. Then it terminates. The next interval spawns a fresh runner. No poller instance persists across cycles.

Processor runners are spawned on system startup or when a new data source connection is configured. They establish their external connection (API subscription, webhook listener, file watcher) and begin processing incoming data. They persist as long as their connection is active. On a configurable interval — or when the runner’s internal turn

count approaches the threshold where LLM attention begins to degrade — the processor snapshots its connection state and session context as knowledge base facts, terminates, and a fresh clone is spawned to resume. The clone reads the snapshot, re-establishes the connection, and continues. From the data source’s perspective, the connection may briefly interrupt; from the knowledge base’s perspective, the transition is seamless.

Internal processing runners are spawned by the same system scheduler that manages pollers, on their own configured interval. They spawn, execute one evaluation cycle across their assigned knowledge base scope, write derived facts and coverage metrics, and terminate. Fresh instance every cycle.

Concurrency Model

All runners share access to the same knowledge base infrastructure. Concurrent access is safe because of structural properties of VDR’s storage model.

Knowledge base writes go through the primitive pipeline, which uses append-only arenas with bump-pointer allocation. Writing a new fact is an atomic pointer increment followed by a data write to the allocated space. Two runners writing to the same knowledge base simultaneously each get their own allocation — no contention, no locking on the write path. Reads see a consistent snapshot defined by the current arena position at the time the read begins.

Queue operations use atomic primitives. Enqueue is an atomic write to the queue’s write position. Dequeue is an atomic read from the queue’s read position. Multiple pollers checking the same queue see consistent state — an item dequeued by one poller is not visible to another.

Counter increments are atomic integer operations. Multiple runners can increment the same counter concurrently without corruption. Counter reads return the current value at the time of the read.

Knowledge base queries (reads) are non-blocking. A query walks the scope chain, checks visibility, and scans the relevant predicate-major column groups. Concurrent writes by other runners may add facts that appear in subsequent queries but do not affect the consistency of an in-progress query. The query sees a consistent snapshot as of its start time.

This concurrency model requires no runner-level locking, no distributed transactions, and no coordination protocols. It inherits from VDR’s append-only arena design: writes never modify existing data, reads see consistent snapshots, and atomic operations on queues and counters provide the necessary synchronization for work distribution.

Resource Allocation

LLM compute is the scarce resource. The scheduler distributes it across runner types with a priority model.

Interactive runners get highest priority. User-facing latency matters — the human is waiting. When an interactive runner needs LLM compute, it preempts batch work.

Processor runners get medium priority. They need to keep up with their data streams to avoid losing data or falling behind. They can buffer briefly but not indefinitely.

Polling runners and internal processors get lowest priority. They are batch operations that can wait for compute availability. If the system is busy serving interactive users and keeping up with data streams, maintenance tasks simply run on the next cycle when compute is available. Their work is never lost — the queues, directories, and coverage gaps will still be there on the next cycle.

The scheduler tracks compute utilization and adjusts runner spawn rates accordingly. If the system is under heavy interactive load, polling intervals may effectively lengthen as pollers wait for compute. If the system is idle, pollers and internal processors consume available compute for maintenance and coverage work. The system naturally shifts between interactive-heavy and batch-heavy operation based on demand.

Grant Scoping Per Runner Type

Grants are the minimum set necessary for each runner type's function.

Interactive runners inherit the authenticated user's grants. They can read knowledge bases the user can read, write to knowledge bases the user can write to, and execute operations the user is granted. They cannot access other users' private knowledge bases, system-level queues, or administrative functions unless the user holds those grants.

Polling runners hold system-level grants for queue management (read and write), directory watching (filesystem read on configured watch paths), task dispatch (write to task queues), and manifest management (write to the processing manifest knowledge base). They do not hold user-level data access grants. They route work — they don't access user data.

Processor runners hold credential grants for their specific external connections (API keys, webhook secrets, stream authentication) and write grants for their designated target knowledge bases. Each processor's grants are scoped to its specific function. A metrics processor has write access to the metrics knowledge base. A document processor has write access to the ingestion knowledge base. Neither can write to the other's target.

Internal processing runners hold broad read grants across project knowledge bases within their evaluation scope, plus write grants for derived facts and coverage metrics. They can read widely to evaluate coverage and consistency, but their write access is limited to their own output — derived facts, metrics, gap descriptions. They cannot modify source facts, retract rules, or change organizational structure. Those operations require owner-level grants.

Health Monitoring

A dedicated internal processing runner monitors system health on each cycle. It checks whether pollers are completing their cycles within expected timeframes (by reading cycle-completion facts that each poller writes). It checks whether processor runners are keeping up with their data streams (by comparing ingestion rate facts against stream rate facts). It checks whether queues are growing or draining (by reading queue depth counters). It

checks whether coverage evaluation cycles are completing (by reading cycle-completion timestamps).

Health metrics are knowledge base facts, queryable through the same mechanisms as any other facts. Alert conditions are Prolog rules: if a queue depth exceeds a threshold, if a poller hasn't completed a cycle within twice its expected interval, if a processor's ingestion rate drops below a fraction of its stream rate. When alert rules fire, the system deposits an alert fact in the review directory for the owner and, if configured, sends a notification through whatever channel the owner has granted.

9. Task Specification Format

Tasks are how work moves through the system — from owner to runners, from runner to runner, from coverage gaps to fetch operations. A task is a structured description of work to be done. The system treats tasks the same way it treats any other structured document: it compacts them into executable knowledge base facts.

Task Structure

A task specifies an action type — what kind of work to do. Fetch: retrieve content from a path or URL. Compact: process a document into knowledge base facts. Reorganize: move facts between knowledge base branches, update scope paths. Analyze: run coverage evaluation, consistency checks, or custom analysis on a knowledge base scope. Report: generate formatted output from knowledge base queries. Export: write knowledge base content to files in specified formats.

A task specifies a target — what to act on. A filesystem path for fetch and compact tasks. A knowledge base path for reorganize and analyze tasks. A query specification for report tasks. An output path and format for export tasks.

A task specifies parameters appropriate to its action type. Depth targets for coverage analysis. Format specifications for exports. Filter criteria for reports. Priority level. Dependencies — other tasks that must complete first.

A task carries metadata: who requested it (owner, poller, internal processor, coverage gap), when it was created, which coverage gap it addresses (if applicable), and what success looks like (which conditions should be true after the task completes).

Task Lifecycle

A task moves through a defined sequence of states.

Created: a file appears in the tasks directory, or a runner enqueues a task into the task queue. The task exists as raw input.

Parsed: the system compacts the task specification into typed knowledge base facts. If the task was written in natural language, the LLM interprets it into structured fields. If it was written in the adjusted compaction format, it parses directly without LLM involvement.

Validated: the system checks whether the requesting identity has grants for the operations the task specifies. A task requesting filesystem access is checked against filesystem grants. A task requesting knowledge base reorganization is checked against write grants on the target knowledge bases. A task that fails validation is rejected — the rejection is logged with the specific grant that was missing, and a notification is placed in the review directory.

Queued: the validated task enters the priority-ordered task queue. Priority is determined by the task's explicit priority field, the depth target of the coverage gap it addresses, and whether it blocks other tasks.

Assigned: the scheduler assigns the task to an appropriate runner. Fetch tasks go to processor runners. Compact tasks go to processor runners. Analysis tasks go to internal processing runners. Report and export tasks go to whichever runner type has the appropriate grants.

Executing: the assigned runner processes the task through the standard VDR primitive pipeline.

Completed: results are stored in the knowledge base with provenance linking them to the task. The task fact is updated with completion status, result references, and timestamp. If the task specified follow-up tasks, those are created and enter the pipeline.

Failed: if execution fails — network error on a fetch, parse error on a compact, missing data for an analysis — the failure is logged with the specific error. The task may be retried based on retry rules (configurable per task type), escalated to the review directory for owner attention, or abandoned with the failure recorded for coverage evaluation to account for.

Task Chaining

A task can declare follow-up tasks that are triggered by its completion. A fetch task completes and triggers a compact task for the fetched content. A compact task completes and triggers a coverage evaluation task for the affected knowledge base branch. A coverage evaluation completes and triggers fetch tasks for identified gaps.

The coverage loop is implemented as task chains. Internal processors create coverage evaluation tasks. Those tasks produce gap descriptions. Gap descriptions trigger fetch tasks. Fetch tasks trigger compact tasks. Compact tasks trigger re-evaluation tasks. The chain continues until coverage targets are met.

Task chains are declared in the task specification as references to follow-up task templates. The templates are knowledge base facts. When a task completes, the system instantiates its follow-up templates with the results of the completed task and enqueues the new tasks. This means the coverage loop doesn't require any special mechanism — it's a chain of standard tasks triggering standard tasks through standard template instantiation.

10. Worked Example: From Empty to Operational

Hour Zero

The seed is loaded. Four layers of operational competence are in the knowledge base: language templates for parsing and generation, format grammars for standard data types, operational rules for primitive composition and pipeline sequencing, and self-maintenance rules for gap detection and lifecycle management. The system can parse input, generate output, invoke primitives through command tokens, and manage its own structures. It has no domain knowledge.

The scheduler is configured but no pollers are assigned to watch directories. No processors have data connections. The system is operational but idle.

Hour One

The owner connects through the chat interface. An interactive runner spawns with the owner's grants.

The owner describes their data layout and organizational goals. They have Project Gutenberg texts in /data/gutenberg, man pages in /usr/share/man, Linux kernel source in /data/repos/linux, Python source and documentation in /data/repos/python, Zig source and documentation in /data/repos/zig.

They want programming as one top-level branch with per-language children. Literature as a second top-level branch organized by period and genre. OS internals as a third branch cross-referencing with man pages and the programming branch where kernel APIs surface.

The LLM proposes a hierarchy as Prolog facts. The owner reviews, makes adjustments — networking should be its own branch because it cross-cuts OS internals, programming, and even some literature. The LLM adjusts. The owner approves. The hierarchy is asserted into the knowledge base.

The owner specifies coverage targets. Python: intermediate, emphasizing standard library API coverage. Zig: intermediate, emphasizing standard library and build system. Linux: focused on memory management and networking subsystems. Literature: broad coverage, period-organized, with cross-references to relevant technical concepts where they exist.

Coverage targets are asserted as knowledge base facts. The owner configures pollers for each data directory and sets polling intervals — every thirty seconds for initial bulk ingestion, slowing to every ten minutes once the backlog is cleared.

The owner says “run it” and disconnects. Total interaction time: approximately fifteen minutes.

Hours Two Through Six

Pollers activate on their configured intervals. Each cycle, a poller checks its assigned directory, finds unprocessed files, and enqueues compaction tasks.

The Gutenberg poller starts walking /data/gutenberg. Hundreds of text files are discovered and enqueued. Processor runners begin compacting. The first few Gutenberg texts require LLM judgment to write the compaction rule for Gutenberg's header format and the grammars for body content extraction. By the fifth text, the header rule handles structural extraction mechanically. Body content extraction still requires LLM judgment for entity and relationship identification but becomes faster as rules accumulate.

The man page poller walks /usr/share/man. Thousands of man page files are enqueued. The compaction rule for man page format is written on the first or second page. After that, structural extraction is mechanical for every man page. The SEE ALSO relationship extraction begins building the command relationship graph.

The Linux source poller walks /data/repos/linux. Multiple file types are detected — .c source files, .h headers, Makefiles, Kconfig files, documentation in various formats. Each type needs its own compaction rule. The LLM writes rules for each type as it encounters them. By the time it has processed a few dozen files of each type, the rules are mature enough to handle structural extraction for most files.

Internal processors begin their evaluation cycles. Coverage is low across all topics — large Remainders, everything is a gap. But facts are accumulating rapidly. The first coverage metrics are written, establishing the baseline for future comparison.

Hours Six Through Twenty-Four

Bulk ingestion continues. Thousands of Gutenberg texts are compacted. The literature knowledge base grows with character, location, event, and theme facts. Cross-reference rules between works begin to accumulate — the system identifies when the same historical figure appears in multiple texts, when the same location is described in different periods, when thematic patterns recur across authors.

The full man page collection is processed. The command relationship graph is comprehensive. API facts from man section 3 are cross-referenced with the programming knowledge base.

The Linux source tree is being processed in depth. Dense cross-referencing between source files, headers, man pages, and documentation is building. The memory management subsystem facts are accumulating as specified in the owner's coverage targets.

Python and Zig documentation and source are being processed. Per-language knowledge bases are filling. Cross-language comparison rules are beginning to form — the system identifies structural similarities between Python's and Zig's standard library organization.

Coverage Remainders are shrinking in well-represented areas. Internal processors identify specific gaps — Python's asyncio module is thin, Zig's build system documentation isn't fully cross-referenced with examples, the Linux networking subsystem is below the coverage target. These gap descriptions appear in coverage metrics.

Day Two Through Seven

The coverage loop is actively driving targeted work. Pollers read coverage metrics, find gaps, and enqueue targeted fetch tasks. If the gap is in local data that hasn't been reached yet, the fetch task points to the specific files. If the gap requires content the local library doesn't have, a gap description appears in the review directory for the owner.

Meta-rules are forming. The system has processed enough programming documentation to write generalized rules about API documentation structure. It has processed enough Gutenberg texts to write generalized rules about literary narrative structure. It has processed enough source code to write generalized rules about repository organization. These meta-rules accelerate processing of remaining documents in each category.

The owner checks in through the chat interface. "How's coverage on Python standard library?" The system reports: 73% of modules have signature-level facts, 45% have example-level coverage, 12 specific modules are below the intermediate threshold, here they are. The owner says "prioritize the networking and async modules." A priority adjustment is asserted. The pollers will shift their fetch task generation accordingly.

The hierarchy has been working well. A few organizational proposals appeared in the review directory — the system wasn't sure whether certain cross-cutting topics (error handling, testing, documentation tools) should be their own branches or sub-topics. The owner makes decisions. The system's organizational rules update.

Week Two Onward

Coverage targets are being met in most areas. The literature knowledge base is broad if not deep. The programming knowledge bases have solid intermediate coverage. The OS internals knowledge base has focused depth on memory management and networking as specified.

The owner adds new targets. "Add Rust, similar depth to Python and Zig." They drop Rust documentation and source in the appropriate directories. The system's generalized programming documentation rules handle much of the compaction with minimal LLM judgment — the meta-rules from processing Python and Zig apply. A Rust-specific compaction rule is written for Rust's documentation format, but the entity extraction and relationship mapping reuse generalized patterns.

The owner is now auditing rather than directing. They spot-check facts, verify cross-references, occasionally correct a classification. The system's organizational proposals match their preferences because the organizational rules have converged through prior corrections. The pollers, processors, and internal processors run autonomously. The knowledge base grows.

11. Security in Multi-Runner Deployment

VDR enforces access control through four structural mechanisms. Knowledge base visibility (public, internal, or owner-only) is checked by integer comparison on every query — unauthorized data never enters any LLM instance's context. Scope chains walk from

the querying session's position upward through the knowledge base tree, with sibling branches structurally unreachable. Grants default to denial on all operations, with positive credential required for each. Output constraints validate content in grammar slots post-generation before rendering. These mechanisms are specified fully in VDR-16.

Every runner type operates through the same primitive pipeline governed by these same mechanisms. Multi-runner deployment does not require additional security mechanisms because it does not introduce additional access paths.

Runner Isolation

Each runner has its own session identity with its own grants. Sessions are established at runner spawn time by the scheduler (for system runners) or by the authentication system (for interactive runners). A poller's session identity is a system identity with system-level grants. It cannot impersonate a user. A processor's session identity is a service identity with grants specific to its data source. It cannot access knowledge bases outside its designated scope.

No runner can escalate its own grants. Grant modification requires administrative grants that no runner type holds in normal operation. A runner that attempts an operation beyond its grants receives a rejection logged in the audit knowledge base with the specific missing grant identified.

Directory Interface Security

Files in the ingress directory are processed through the same compaction pipeline as any other input. The compaction pipeline operates through primitives that check grants and log operations. A malicious file — one containing content designed to manipulate the LLM into unauthorized operations — is limited by the same structural constraints that limit all LLM behavior. The LLM can attempt to issue command tokens for unauthorized operations. Those command tokens go through grant verification in the primitive layer. Unauthorized operations are rejected. The LLM's intent is irrelevant to the access control outcome because access control operates on integer values set at session establishment, not on anything the LLM generates.

The ingress directory itself is accessible only through filesystem grants held by the designated poller. Other runners cannot read from or write to the ingress directory. The output and review directories follow the same pattern — specific grants for specific runners, no shared ambient access.

Task Validation

Every task is validated against the requesting identity's grants before execution. When a poller enqueues a task generated from a coverage gap, the task is tagged with the poller's identity. When the task is assigned to a processor for execution, the processor's grants are checked against the operations the task requires. If the task requires filesystem access the processor isn't granted, the task is rejected.

This means the coverage loop cannot be used to escalate access. Even if an internal processor identifies a “gap” that would require accessing restricted knowledge bases, the fetch task generated from that gap fails validation because no runner in the pipeline holds grants for the restricted data. The gap persists in the coverage metrics, visible to the owner, who can address it through their own grants if appropriate.

Cross-Runner Communication

All communication between runners goes through knowledge bases and queues. There is no direct runner-to-runner channel. A poller enqueues a task by writing to a queue through the primitive pipeline. A processor dequeues by reading from the queue through the primitive pipeline. Both operations go through grant checks. Both are logged.

This means runners cannot coordinate to bypass access controls. Two runners cannot collude to access data that neither is individually granted, because every data access goes through the per-session visibility and scope checks regardless of how the access was initiated. The structural isolation that prevents the LLM from bypassing access control in a single session prevents multiple LLM instances from bypassing it collectively.

12. Scaling

Data Volume

More data means more knowledge base facts at integer addresses. Individual fact lookup remains constant time through indexed access. Predicate-major columnar storage means queries scan only the relevant predicate’s column group, not the entire knowledge base. Rule evaluation scales with the size of the fact base within the queried scope, but VDR-18’s frontier-based GPU evaluator handles large-scale rule evaluation efficiently through batched joins.

Storage grows linearly with fact count. Each fact is stored at a fixed-width entry in its predicate’s column group with provenance metadata. The append-only arena allocation means no fragmentation — storage utilization is near-optimal.

Runner Count

More runners means more concurrent processing. Knowledge base access is naturally concurrent through the append-only arena model. Queue-based work distribution means runners don’t compete for tasks — each task is dequeued exactly once through atomic operations. The scheduler balances compute across runner types according to the priority model.

Adding runners is straightforward: configure additional pollers for additional directories, spawn additional processors for additional data streams, increase internal processor frequency for faster coverage evaluation. Each additional runner gets its own session, grants, and scope. No existing runner is affected.

Knowledge Growth

As the knowledge base grows and the rule base matures, the system requires less LLM judgment per unit of work. Compaction rules handle structural extraction. Classification rules handle document routing. Organizational rules handle hierarchy management. Coverage rules handle gap identification. The LLM's judgment is reserved for genuinely novel situations.

This means the compute cost per ingested document decreases over time. Early in the system's life, each document requires significant LLM judgment for compaction rules and entity identification. After processing hundreds of documents of similar types, the LLM's role shrinks to confirming what rules propose and handling exceptions. The system gets more efficient as it grows.

Multi-Owner Deployment

Multiple owners can share the same VDR infrastructure with separate scope trees. Each owner's knowledge bases are visibility-isolated — owner-only knowledge bases are invisible to other owners. Shared public knowledge bases (standard library documentation, open source code facts, general reference material) are readable by all owners and writable only by system-level runners with appropriate grants.

Organizational rules at the system level (document type classification, format grammars, meta-rules about document structure) apply universally and benefit all owners. Each owner's domain-specific rules, coverage targets, and hierarchy customizations are scoped to their branch. An owner benefits from the system's growing competence at document processing without seeing or being affected by another owner's domain-specific knowledge.

Appendix A: Prompt Runner Specification

A.1 — Runner Type Comparison

Property	Interactive	Polling	Processor	Internal Processing
Trigger	User input	Timer interval	Data arrival on stream	Scheduler interval
Lifecycle	Session duration	One cycle	Long-lived with respawn	One cycle
Typical duration	Minutes to hours	Seconds	Hours to days (with respawns)	Seconds to minutes
LLM freshness	Fresh per session	Fresh per cycle	Fresh per respawn	Fresh per cycle
Compute priority	Highest	Lowest	Medium	Lowest

Property	Interactive	Polling	Processor	Internal Processing
Session identity	Authenticated user	System poller identity	Service identity	System evaluator identity
KB read grants	User's visible scope	System queues + manifests	Designated source KBs	Broad project read scope
KB write grants	User's writable scope	Queue + manifest write	Designated target KBs	Derived facts + metrics
External access	Via user's grants	Filesystem watch paths	Credentialed connections	None
Typical token count per activation	50–500	10–50	8–30 per item	20–100

A.2 — Runner Grant Matrix

Operation	Interactive	Polling	Processor	Internal	Owner (chat)
Read user project KB	User's scope	No	No	Read only	Full
Write user project KB	User's scope	No	Designated only	Derived facts only	Full
Read system queues	No	Yes	Task queue only	No	Yes
Write system queues	Task enqueue	Task dispatch	Result enqueue	Gap enqueue	Full
Filesystem read	Via user grant	Watch paths only	Source paths only	No	Full
Filesystem write	Via user grant	Manifest only	No	No	Full
External network	Via user grant	No	Credentialed only	No	Via owner grant
Docker execution	Via user grant	No	No	No	Full
Grant modification	No	No	No	No	Admin grant required
KB creation	Via user grant	No	No	No	Full

Operation	Interactive	Polling	Processor	Internal	Owner (chat)
Rule assertion	User's writable scope	No	Designated KB only	Derived rules only	Full
Rule retraction	User's writable scope	No	No	No	Full

A.3 — Clone Lifecycle Per Runner Type

Event	Interactive	Polling	Processor	Internal
Spawn trigger	User connects	Scheduler interval	Startup / config change	Scheduler interval
Session setup	Auth → scope → grants	System identity → grants	Service identity → grants → connect	System identity → grants
KB snapshot on spawn	No (session-scoped)	No (stateless per cycle)	Yes (resume state)	No (stateless per cycle)
Work pattern	Request-response	Check-dispatch-terminate	Continuous ingest	Evaluate-write-terminate
Respawn trigger	N/A (session-bound)	Next interval	Turn count threshold	Next interval
Snapshot on termination	Promote selected findings	Write cycle-complete fact	Snapshot connection + session state	Write metrics + cycle-complete
Typical turns per lifecycle	1–100	1–5	20–50 (then respawn)	5–20

Appendix B: Directory Interface Specification

B.1 — Directory Conventions

Directory	Purpose	Watched By	Write Access	Read Access
/vdr/ingress/	Owner drops files for compaction	Ingress poller	Owner (external)	Ingress poller
/vdr/tasks/	Owner drops task specifications	Task poller	Owner (external)	Task poller

Directory	Purpose	Watched By	Write Access	Read Access
/vdr/config/	Owner drops configuration changes	Config poller	Owner (external)	Config poller
/vdr/output/	System deposits results	Output processor	Designated runners	Owner (external)
/vdr/review/	System deposits items for owner judgment	Review processor	Designated runners	Owner (external)
/vdr/manifests/	Processing records	Manifest writer	Pollers + processors	All runners (read)

B.2 — File Routing Rules

File Extension	Detected Type	Default Target KB Branch	Compaction Pipeline
.txt	Plain text	Classification-dependent	Classify → route → compact
.md	Markdown	Classification-dependent	Markdown grammar → extract → classify → route
.json	JSON data	Classification-dependent	JSON grammar → parse → classify → route
.csv	Tabular data	Classification-dependent	CSV grammar → parse → classify → route
.py	Python source	root.programming.python	Python compaction rules → extract
.zig	Zig source	root.programming.zig	Zig compaction rules → extract
.c / .h	C source/header	Classification-dependent	C compaction rules → extract → cross-ref
.pdf	PDF document	Classification-dependent	PDF extract → text → classify → route
.html	Web page	Classification-dependent	HTML extract → text → classify → route
.pl / .pro	Prolog source	Direct load	Parse as Prolog clauses → assert directly

File Extension	Detected Type	Default Target KB Branch	Compaction Pipeline
.task	Task specification	Task queue	Parse → validate → enqueue
.config	Configuration	System config KB	Parse → validate → assert

B.3 — Manifest Entry Schema

Field	Type	Description	Example
file path	string	Original filesystem path	/data/gutenberg/pg1342.txt
file hash	integer	Content hash for change detection	8847291043
detected type	atom	Classification result	gutenberg text
compaction rule	atom	Rule used for extraction	compact gutenberg v3
target kb	path	Where facts were stored	root.literature.19th century.novel
facts extracted	integer	Count of facts produced	347
rules written	integer	Count of rules produced	4
processed at	integer	Timestamp	1716000000
processor session	integer	Which runner processed it	session 4471
status	atom	Processing result	completed / failed / partial

Appendix C: Coverage Metric Specification

C.1 — Coverage Metric Structure

Field	Type	Description	Example
topic	path	KB path for the topic	root.programming.python.stdlib
depth target	atom	Specified depth level	intermediate
total target items	integer	Items needed for target	312 (modules × coverage aspects)
covered items	integer	Items with sufficient facts	228

Field	Type	Description	Example
coverage value	Q335	Covered / total as exact fraction	228/312
remainder count	integer	Number of specific gaps	84
remainder items	list(ref)	References to gap descriptions	[gap 4401, gap 4402, ...]
evaluated at	integer	Timestamp of evaluation	1716003600
evaluator session	integer	Which internal processor	session 4523

C.2 — Gap Description Schema

Field	Type	Description	Example
gap id	integer	Unique identifier	gap 4401
topic	path	Parent topic	root.programming.python.stdlib
missing item	string	What's absent	“asyncio event loop patterns”
gap type	atom	Category of gap	api coverage / examples / cross reference / error handling
priority	integer	Derived from depth target and gap type	2
suggested source	path or URL	Where to find it	/data/repos/python/Doc/library/asyncio-eventloop.rst
blocking	list(ref)	Other gaps this blocks	[gap 4415, gap 4416]
created at	integer	When identified	1716003600
addressed by	ref or null	Task created to fill this gap	task 7802 or null

C.3 — Coverage Rule Examples

Rule	Evaluates	Threshold	Fires When
module signature coverage(Module, Score)	Facts with predicate function signature in Module KB	Score = count / total public functions	Always — produces metric

Rule	Evaluates	Threshold	Fires When
module example coverage(Module, Score)	Facts with predicate usage example in Module KB	Score = count / total public functions	Always — produces metric
cross reference density(Module, Score)	Relationship facts linking Module to other modules	Score = relationships / total public functions	Always — produces metric
intermediate target met(Module)	All three scores above intermediate thresholds	Signature > 0.9, Example > 0.5, CrossRef > 0.3	When module meets intermediate
gap identified(Module, Type)	Score below threshold for specific coverage type	Per-type threshold from depth definition	When gap exists — triggers task

C.4 — Remainder Decomposition Example

Coverage Query	Value	Remainder (decomposed)
Python stdlib overall	228/312 = 73.1%	84 modules below intermediate threshold
Python stdlib signatures	289/312 = 92.6%	23 modules missing signatures: [asyncio.runners, asyncio.tasks, ...]
Python stdlib examples	156/312 = 50.0%	156 modules missing examples: [most non-core modules]
Python stdlib cross-refs	94/312 = 30.1%	218 modules missing cross-references: [most modules]
Python asyncio specifically	3/18 = 16.7%	15 asyncio sub-items: [event loop, tasks, streams, queues, ...]

Appendix D: Task Lifecycle State Machine

D.1 — Task States and Transitions

From State	To State	Trigger	Action
—	created	File in tasks dir / runner enqueue	Raw task exists

From State	To State	Trigger	Action
created	parsed	System compacts task spec	Typed task facts in KB
parsed	validated	Grant check passes	Task cleared for execution
parsed	rejected	Grant check fails	Logged with missing grant; review item created
validated	queued	Priority assigned	Task in priority queue
queued	assigned	Scheduler selects runner	Runner bound to task
assigned	executing	Runner begins work	Primitives invoked
executing	completed	Work finished successfully	Results stored; follow-ups triggered
executing	failed	Error during execution	Error logged; retry or escalate
failed	queued	Retry rule fires	Re-enters queue with retry count incremented
failed	abandoned	Max retries exceeded	Logged; review item if configured
completed	—	Follow-up tasks created if specified	Chain continues

D.2 — Task Chaining Templates

Trigger Task Type	Follow-up Task Type	Instantiation	Example
fetch	compact	Target = fetched file path	Fetch asyncio docs → compact them
compact	coverage evaluate	Target = affected KB branch	Compact Python module → re-evaluate Python coverage
coverage evaluate	fetch (per gap)	One task per gap with priority	Evaluate Python → fetch for each gap
reorganize	coverage evaluate	Target = reorganized branch	Move KB branch → re-evaluate coverage

Trigger Task Type	Follow-up Task Type	Instantiation	Example
compact	cross reference	Target = newly compacted KB + related KBs	Compact man page → cross-ref with source KB

D.3 — Task Priority Calculation

Factor	Weight	Source	Example
Owner-specified priority	Highest	Task file or config	“prioritize async modules” → priority 1
Coverage gap severity	High	Remainder size relative to target	15/18 asyncio items missing → high priority
Blocking count	Medium	How many other gaps this blocks	Gap blocks 3 others → elevated priority
Depth target level	Medium	Topic specification	Intermediate target → higher than broad-survey target
Task type	Low	Default per type	Compact > cross-reference > report
Age	Tiebreaker	Time since creation	Older tasks break ties

Appendix E: Thread Concurrency Model

E.1 — Concurrent Access Patterns

Resource	Read Pattern	Write Pattern	Contention Model
KB fact table	Non-blocking snapshot read	Atomic bump-pointer append	Zero contention — reads and writes independent
KB predicate index	Non-blocking lookup	Append on new fact	Minimal — index update is atomic
Task queue	Atomic dequeue	Atomic enqueue	Zero — separate read/write positions
Counter	Atomic read	Atomic increment	Zero — single instruction

Resource	Read Pattern	Write Pattern	Contention Model
Coverage metrics KB	Non-blocking read	Periodic bulk write by internal processor	Minimal — writes are infrequent
Manifest KB	Non-blocking read	Append per processed file	Zero — append-only
Audit KB	Non-blocking read (rare)	Append per operation	Zero — append-only, write-heavy
Arena allocator	N/A	Atomic bump pointer	Zero — one instruction per allocation

E.2 — Consistency Guarantees

Scenario	Guarantee	Mechanism
Two runners write to same KB simultaneously	Both writes succeed, both visible to future reads	Append-only — no overwrite, no corruption
Runner reads KB while another writes	Reader sees consistent snapshot as of read start	Arena position at read start defines visible facts
Two pollers check same queue	Each item dequeued exactly once	Atomic read position — dequeue is compare-and-swap
Coverage evaluation during active ingestion	Evaluation reflects facts present at cycle start	Snapshot consistency — new facts appear in next cycle
Runner crashes mid-write	Partial write not visible to readers	Arena position advances only on complete write
Multiple internal processors evaluate same KB	Each produces independent metrics	No write contention — each writes to own metric facts

Appendix F: Worked Example Timeline

F.1 — Hour-by-Hour Metrics

Time	Files Discovered	Files Compacted	KB Facts	Prolog Rules	Compaction Rules	Coverage (Python)	Coverage (Lit)
Hour 0	0	0	23,398 (seed)	368 (seed)	0	0%	0%
Hour 1	0	0	23,398	368	0	0%	0%

Time	Files Discov- ered	Files Com- pacted	KB Facts	Prolog Rules	Compaction Rules	Coverage (Python)	Coverage (Lit)
Hour 2	2,400	45	25,100	380	6	3%	1%
Hour 4	8,500	320	38,000	425	14	12%	5%
Hour 8	15,000	1,800	72,000	510	22	28%	12%
Hour 12	18,000	4,200	130,000	620	28	41%	22%
Hour 24	22,000	9,500	280,000	780	35	58%	38%
Day 3	25,000	18,000	520,000	950	42	73%	55%
Day 7	27,000	24,000	780,000	1,100	48	85%	68%
Day 14	28,000	27,000	1,050,000	1,250	52	92%	78%
Day 30	29,000	28,500	1,300,000	1,400	55	96%	85%

F.2 — Token Efficiency Over Time

Time	Avg Tokens per Compaction	LLM Judgment %	Rule-Handled %	Meta-Rule Assists
Hour 2	180	85%	15%	0
Hour 8	95	55%	45%	2
Hour 24	52	30%	70%	8
Day 3	38	20%	75%	15
Day 7	28	12%	82%	22
Day 14	22	8%	85%	28
Day 30	18	5%	88%	32

F.3 — Runner Activity Distribution

Time	Interactive	Polling Cycles/hr	Processor Active	Internal Cycles/hr
Hour 0	0	0	0	0
Hour 1	1 (owner setup)	0	0	0
Hour 2	0	120	3	12
Hour 8	0	120	5	12
Hour 24	1 (owner check-in)	120	5	12
Day 3	1 (owner adjustment)	60 (interval lengthened)	4	6
Day 7	0	30	3	6
Day 14	1 (owner audit)	12	2	4

Time	Interactive	Polling Cycles/hr	Processor Active	Internal Cycles/hr
Day 30	0	6 (maintenance pace)	1	2

Appendix G: Local Data Source Characteristics

G.1 — Source Profiles

Source	Typical Volume	File Count	Format Uniformity	Cross- Reference Density	Compaction Rules Needed
Project Gutenberg	5–50 GB	60,000+ texts	High (standard headers)	Medium (author/period/then- action)	2–3 base + literary action
Unix man pages	50–200 MB	5,000– 15,000 pages	Very high (standard sections)	High (SEE ALSO explicit)	1–2 (format nearly uniform)
Linux kernel source	1–2 GB	50,000+ files	Low (many file types)	Very high (code cross-refs)	8–12 (per file type)
Python source + docs	200–500 MB	10,000+ files	Medium	High (module cross-refs)	4–6
Zig source + docs	100–300 MB	5,000+ files	Medium	High	4–6
Open source textbooks	Variable	Variable	Low (per-book format)	Medium	2–4 per format

G.2 — Expected KB Output Per Source

Source	Facts per 1K Files	Relationships per 1K Files	Meta-Rules Generated	Processing Time (est.)
Project Gutenberg	~15,000	~3,000	Literary structure rules	~4 hrs / 1K texts
Unix man pages	~8,000	~5,000 (SEE ALSO dense)	Command pattern rules	~1 hr / 1K pages
Linux kernel	~25,000	~12,000	Code structure rules	~8 hrs / 1K files

Source	Facts per 1K Files	Relationships per 1K Files	Meta-Rules Generated	Processing Time (est.)
Python source + docs	~20,000	~8,000	API documentation rules	~3 hrs / 1K files
Zig source + docs	~18,000	~7,000	API documentation rules	~3 hrs / 1K files

Appendix H: Hierarchy Proposal and Convergence

H.1 — Initial vs Converged Hierarchy Example

Data Source	Filesystem-Derived Hierarchy	Owner-Converged Hierarchy	Rules Written
/data/gutenberg/*.txt	root.sources.gutenberg	root.literature.[period] [genre]	classify by period, classify by genre
/usr/share/man/man1/*	root.sources.man.man1	root.os.commands.* + cross-ref root.programming	man section to branch, api cross ref
/usr/share/man/man3/*	root.sources.man.man3	root.programming.c.stdlib	stdlib to programming
/data/repos/linux/mm/*	root.sources.linux.mm	root.os.internals.memory	source subsystem to branch
/data/repos/linux/net/*	root.sources.linux.net	root.networking.kernel	networking cross cut rule
/data/repos/python/Lib/*	root.sources.python.lib	root.programming.pythonstdlib	pythonlib to programming
/data/repos/zig/lib/std/*	root.sources.zig.std.*	root.programming.zig.stdlib	ziglib to programming

H.2 — Convergence Metrics

Owner Corrections	Proposal Accuracy	New Rules From Corrections	Cumulative Organizational Rules
0 (initial)	~40% (filesystem-derived)	0	Seed layer 3 only
5	~55%	8	Seed + 8
10	~70%	14	Seed + 14
20	~82%	22	Seed + 22
50	~93%	35	Seed + 35
100	~97%	42	Seed + 42

Owner Corrections	Proposal Accuracy	New Rules From Corrections	Cumulative Organizational Rules
200+	~99%	45 (plateau)	Seed + 45 (stable)

Appendix I: Runner Health Monitoring

I.1 — Health Metrics

Metric	Source	Normal Range	Alert Threshold	Alert Rule
Poller cycle time	cycle complete facts	<5 sec	>30 sec or missing	poller slow(Poller) :- cycle time(Poller, T), T > 30.
Queue depth (tasks)	Queue counter	0–50	>200	queue backlog(Q) :- depth(Q, D), D > 200.
Queue depth (coverage gaps)	Queue counter	0–100	>500	gap backlog(Q) :- depth(Q, D), D > 500.
Processor throughput	items per minute counter	Source-dependent	<50% of stream rate	processor behind(P) :- rate(P, R), stream rate(P, S), R < S * 0.5.
Internal processor cycle time	cycle complete facts	<60 sec	>300 sec	evaluator slow(E) :- cycle time(E, T), T > 300.
KB growth rate	Fact count delta per hour	Source-dependent	Zero for >2 hours during active ingestion	ingestion stalled :- growth rate(0), active tasks > 0.
Compaction error rate	Failed compaction count / total	<5%	>15%	compaction degraded :- error rate(R), R > 0.15.

Metric	Source	Normal Range	Alert Threshold	Alert Rule
Runner respawn rate	Respawn events per hour	Processor-dependent	$>2 \times \text{expected}$	excessive respawns(R) :- respawn rate(R, N), expected(R, E), $N > E * 2$.

I.2 — Health Response Actions

Alert	Severity	Automatic Response	Owner Notification
Poller slow	Low	Log; continue	Review item if persistent
Queue backlog	Medium	Increase processor priority	Review item
Processor behind	Medium	Spawn additional processor if compute available	Review item
Evaluator slow	Low	Log; may indicate KB growth requiring index optimization	Review item if persistent
Ingestion stalled	High	Check for failed tasks; retry queue	Immediate review item
Compaction degraded	High	Flag affected files for re-processing	Immediate review item
Excessive respawns	Medium	Check processor connection stability	Review item

Appendix J: Security Matrix

J.1 — Operation Authorization by Runner Type

Operation	Interactive	Polling	Processor	Internal	API Client
KB query (own scope)	Granted	System scope	Designated KBs	Evaluation scope	Client scope
KB query (cross-scope)	Denied	Denied	Denied	Read across projects	Denied
KB assert (own scope)	Granted	Manifest only	Designated KBs	Derived facts only	Client scope

Operation	Interactive	Polling	Processor	Internal	API Client
KB retract	Granted (own facts)	Denied	Denied	Denied	Denied
Queue enqueue	Task queue	Task dispatch	Result queue	Gap queue	Task queue
Queue dequeue	Denied	Task + coverage queues	Task queue	Denied	Denied
Filesystem read	User grant	Watch paths	Source paths	Denied	Denied
Filesystem write	User grant	Manifest dir	Denied	Denied	Denied
Network fetch	User grant	Denied	Credentialed	Denied	Client grant
Docker execute	User grant	Denied	Denied	Denied	Client grant
Prolog rule assert	Own scope	Denied	Designated KB	Derived rules	Client scope
Coverage metric write	Denied	Denied	Denied	Granted	Denied
Health metric write	Denied	Cycle-complete only	Throughput only	Health evaluator only	Denied

J.2 — Attack Surface Analysis

Attack Vector	Target	Runner Type Exposure	Structural Defense
Malicious file in ingress	Compaction pipeline	Processor	LLM judgment bounded by grants; command tokens validated; no grant escalation
Crafted task file	Task execution	Via poller to processor	Task validated against requester grants before execution
Coverage gap manipulation	Fetch unauthorized data	Internal → poller → processor	Fetch task validated against processor grants; restricted KBs unreachable

Attack Vector	Target	Runner Type Exposure	Structural Defense
Cross-runner collusion	Collective access escalation	Any combination	Per-session visibility checks on every fact access; composition cannot bypass
Queue poisoning	Inject unauthorized tasks	Any with queue write	Tasks validated at execution, not just creation; executor grants checked
Processor credential theft	Use processor's API credentials	Other runners	Credentials in processor's grant set only; other runners lack access
Health metric spoofing	Hide system problems	Runners writing health facts	Health evaluator cross-checks multiple sources; anomalies flagged

Appendix K: Scaling Projections

K.1 — Data Volume Scaling

KB Facts	Storage (est.)	Query Latency	Coverage Eval Time	Rule Eval Time
100,000	~50 MB	<1 ms	~2 sec	~500 ms
500,000	~250 MB	<1 ms	~8 sec	~2 sec
1,000,000	~500 MB	<1 ms	~15 sec	~4 sec
5,000,000	~2.5 GB	<2 ms	~60 sec	~15 sec
10,000,000	~5 GB	<2 ms	~120 sec	~30 sec
50,000,000	~25 GB	<5 ms (index-dependent)	~10 min	~2 min

K.2 — Runner Count Scaling

Concurrent Runners	Compute Requirement	Queue Throughput	KB Write Throughput	Recommended Hardware
5 (minimal)	1 GPU + CPU	~100 tasks/hr	~10K facts/hr	Single workstation
10 (standard)	1–2 GPU + CPU	~500 tasks/hr	~50K facts/hr	Workstation or small server

Concurrent Runners	Compute Requirement	Queue Throughput	KB Write Throughput	Recommended Hardware
25 (production)	2–4 GPU + CPU	~2K tasks/hr	~200K facts/hr	Server
50 (heavy)	4–8 GPU + CPU	~5K tasks/hr	~500K facts/hr	Multi-GPU server
100+ (enterprise)	GPU cluster	~10K+ tasks/hr	~1M+ facts/hr	Cluster deployment

K.3 — Knowledge Efficiency Over Time

System Age	Avg Tokens per Document Compaction	% Handled by Rules	New Rules per 100 Documents	Meta-Rules Available
Day 1	180	15%	15	0
Week 1	52	70%	5	8
Month 1	18	88%	2	32
Month 3	12	93%	0.5	45
Month 6	10	95%	0.2	52
Year 1	8	97%	0.1	58

Appendix L: API Interface Specification

L.1 — API Operations

Endpoint	Method	Authentication	Grant Required	Description
/query	POST	Session token	KB read (scope-checked)	Query knowledge base facts
/assert	POST	Session token	KB write (scope-checked)	Assert new facts
/task	POST	Session token	Task enqueue	Submit a task for execution
/task/{id}/status	GET	Session token	Task read	Check task status
/coverage/{topic}	GET	Session token	Coverage KB read	Read coverage metrics
/output/{id}	GET	Session token	Output read	Retrieve task output
/health	GET	System token	Health KB read	System health metrics

Endpoint	Method	Authentication	Grant Required	Description
/session	POST	Auth credentials	None (creates session)	Establish authenticated session

L.2 — API Security Model

Property	Implementation	Mechanism
Authentication	Token-based session	Session KB fact with user id, expiry
Authorization	Per-operation grant check	Same grant system as all other runners
Scope isolation	Session scope from user position	Same scope chain as interactive runners
Audit	Every API call logged	Same audit KB as all other operations
Rate limiting	Counter per session per interval	Counter primitive with threshold rule

Appendix M: Owner Workflow Patterns

M.1 — Workflow Comparison

Workflow	Planning Tool	Execution Path	Best For
Chat-driven	VDR interactive runner	Owner describes → LLM proposes → owner approves → facts asserted	Initial setup, hierarchy design, ad-hoc adjustments
File-driven	Any text editor	Owner writes config/task file → drops in directory → poller picks up	Batch configuration, scripted workflows, reproducible setups
External LLM assisted	Claude, GPT, local model	Owner brainstorms externally → saves result → drops in tasks/config dir	Complex planning, strategy development, knowledge organization research

Workflow	Planning Tool	Execution Path	Best For
Hybrid	Multiple tools	Owner plans with external LLM + web research → saves locally → ingests plans + references → uses chat to refine → executes	Full-scale deployment, ongoing management
Fully autonomous	N/A (system-directed)	Coverage loop identifies gaps → generates tasks → fetches → compacts → re-evaluates	Mature system with established targets and organizational rules

M.2 — Owner Time Investment Over System Lifetime

Phase	Duration	Owner Hours/Week	Primary Activities
Initial setup	Day 1	2–4	Hierarchy design, coverage targets, directory configuration
Active bootstrap	Days 2–7	3–5	Hierarchy corrections, coverage adjustments, review items
Guided growth	Weeks 2–4	1–2	Audit spot-checks, priority adjustments, new topic additions
Autonomous operation	Month 2+	0.5–1	Periodic audits, occasional corrections, new data source additions
Mature system	Month 6+	0.25–0.5	Rare corrections, strategic direction changes only

[[HOWL-VDR-20-2026](https://github.com/ghowland/papers/tree/main/papers/VDR/H)] Operational Deployment. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/H>

VDR-20-2026

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-15-2026](#)] VDR-LLM-Prolog: Prompt Optimization. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-15-2026>

[[HOWL-VDR-16-2026](#)] VDR-LLM-Prolog: Safe by Contract. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-16-2026>

[[HOWL-VDR-17-2026](#)] VDR-LLM-Prolog: Alignment. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-17-2026>

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-18-2026>

[[HOWL-VDR-19-2026](#)] VDR-LLM-Prolog: Self-Extending Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-19-2026>